Walmart ✦

QA Notes & Escalation Tool

Improvement Report

September 12, 2026

Improvement Report

Document purpose

This report explains the improvements made to the **QA Notes & Escalation Tool** (`development/programs/All_In_1_v7.html`). It is written for both everyday users and technical reviewers.

The tool remains a single HTML file that can be opened directly in a browser. The work focused on making it safer, more dependable, easier to use, and easier to maintain without changing the established note templates or escalation wording.

> **In plain language:** The tool now does a better job protecting entered information, preventing accidental data loss, explaining errors, supporting keyboard users, and confirming whether copy and file actions succeeded.

At a glance

Area	What was improved	Why it was needed
Security	User-entered escalation text is rendered safely	Entered HTML could previously become active page content
Undo and clearing	Clear and Undo now behave consistently	Undo could become hidden, and clearing notes could erase unrelated escalation work
Validation	Conditional and required fields are enforced consistently	Some incomplete forms could be generated, especially Point Change requests
Clipboard	Copy success and failure are shown clearly	A failed browser copy could previously look successful or still affect the log

Area	What was improved	Why it was needed
Notes Log	Permissions, errors, dates, and file rollover are handled	File failures were mostly invisible and an old daily log could continue being used
Accessibility	Labels, errors, keyboard controls, focus, and dialog behavior were improved	Several controls were difficult or impossible to use without a mouse or with assistive technology
Responsive layout	The QA checklist becomes an overlay in narrow windows	Expanding it could leave too little room for the form
Maintainability	Shared state and repeated data were simplified	Hidden state on page elements made future changes harder to reason about
Testing	Automated coverage now checks every template and escalation type	The app previously had no focused regression safety net

User-facing improvements

1. Safer escalation summaries

What changed

Text entered into escalation fields is now displayed as plain text in the generated summary. Characters that resemble HTML, such as angle brackets, are shown literally rather than interpreted by the browser.

Why this was needed

The former summary builder inserted field values as HTML. A specially formed value could create an unwanted element or run browser behavior inside the page. Even when no one intended to enter code, ordinary values containing symbols could be rendered incorrectly.

Benefit to users

- Entered notes appear as written.
- Unexpected page content cannot be created from escalation fields.
- The copied summary remains plain, predictable text.

Technical detail

The summary renderer now creates DOM elements and assigns user values through `textContent`. Undo stores structured summary rows rather than a block of generated HTML.

2. Reliable Clear and Undo behavior

What changed

- **Escalation Undo** remains visible after an escalation is cleared.
- Undo restores the selected escalation type, entered fields, conditional controls, and generated summary.
- **Clear Notes** only clears the notes workflow.
- Clearing notes no longer resets an in-progress escalation or shared escalation data.
- Undo rebuilds the note preview from restored fields instead of restoring a separate cached display.

Why this was needed

The previous escalation Undo button was located inside the same section that Clear hid, making Undo unreachable. Clearing notes also reset unrelated escalation work, while its Undo action restored only the notes. This created a risk of accidental data loss.

Benefit to users

Clear actions now affect only the section where they were selected, and the matching Undo action restores the complete state users expect.

Technical detail

Escalation and note snapshots now have separate lifecycles. Escalation snapshots contain field values, summary rows, the title, and visibility state. Note snapshots contain the selected template, field values, and user-edited state.

3. More accurate form validation

What changed

- Point Change requests now require **New Board** when **New Board** is selected.
- Switching back to **Same Board** hides and clears the no-longer-applicable value.
- Required fields expose clear visual and screen-reader error states.
- The first incomplete field receives keyboard focus after validation fails.
- Required note fields and escalation fields follow the same validation behavior.

Why this was needed

The New Board value was shown conditionally but remained optional in validation. This allowed an incomplete Point Change request to be generated. In other forms, users could see a red border but receive little help locating or understanding the first problem.

Benefit to users

- Fewer incomplete escalation requests.
- Faster correction when a required value is missing.

- Hidden values do not accidentally appear in generated output.

Technical detail

Conditional requirements are defined in field metadata through `requiredWhen`. A shared validity helper updates the CSS state, `aria-invalid`, and the associated error message together.

4. Predictable DAE routing

What changed

Refrigeration Troubleshooting and **Comm Loss Troubleshooting** automatically select and lock **Digital Assets L3/Docfit**. Selecting another DAE type unlocks the routing field and clears the previously forced value.

Why this was needed

The previous behavior forced L3 for two DAE types but did not reliably undo that choice after the type changed. A stale routing value could therefore be carried into a different request.

Benefit to users

The routing choice always matches the current DAE type and remains editable when policy does not require a fixed destination.

5. Clear clipboard feedback

What changed

All copy actions now use one shared clipboard process. The tool:

- Confirms successful copies.
- Shows a visible message when browser permission or clipboard access fails.
- Restores button labels after confirmation.
- Writes to the Notes Log only after the matching copy succeeds.

Why this was needed

Clipboard operations can fail because of browser permissions or security settings. Previously, errors were not handled consistently, and a failed copy could still lead to a log entry. That could make the daily record disagree with what actually reached the clipboard.

Benefit to users

Users receive an honest result and can retry immediately when browser access is blocked.

Technical detail

`writeClipboard()` is now the single asynchronous adapter used by note, escalation, field, and Helpdesk copy actions. It returns a success result that controls whether later actions, such as file logging, may continue.

6. More dependable Notes Log files

What changed

The Notes Log now:

- Checks that the browser supports the required file APIs.
- Clearly states that current Chrome or Edge is required when support is missing.
- Verifies read/write permission before using a selected file.
- Reports canceled pickers, denied permissions, read failures, and write failures.
- Uses unambiguous `YYYY-MM-DD` dates in new filenames and entries.
- Detects when a selected daily log belongs to a previous date.
- Lets the user continue with the old file or select a new daily file.
- Warns when a log exceeds 5 MB and may take longer to save.

Why this was needed

File errors previously appeared only in the browser console, where most users would never see them. The app also remembered the date of a selected file but did not use it, so later entries could silently continue into an older daily log.

Benefit to users

The visible status now reflects what happened to the file, and daily records are less likely to be written to the wrong log.

Technical detail

The tool capability-checks `showOpenFilePicker` and `showSaveFilePicker`, checks or requests `readwrite` permission, and returns explicit success values from file operations. The existing full-file rewrite approach was retained for standalone compatibility; the 5 MB warning makes its practical limit visible.

Accessibility and usability improvements

7. Labels and errors work with assistive technology

What changed

- Every generated input has a matching programmatic label.

- Required controls expose `aria-required`.
- Error descriptions are connected through `aria-describedby`.
- Invalid controls expose `aria-invalid`.
- Live preview, copy results, and Notes Log results use live status regions.
- The generated escalation summary is identified as a labelled region.

Why this was needed

Visible labels were present, but many were not technically connected to their fields. Screen readers could not reliably identify a field, its requirement, or its error. Visual users also benefited from focus moving directly to the first issue.

Benefit to users

The forms are clearer for screen-reader users, keyboard users, and anyone correcting a long escalation form.

8. Keyboard-accessible QA checklist

What changed

- Checklist items are native checkboxes rather than clickable page containers.
- The checklist toggle is a real button.
- Expanded and collapsed state is exposed to assistive technology.
- Reset clears both the visual style and checkbox state.
- Escape closes an open checklist and returns focus to its toggle.

Why this was needed

Clickable `div` elements do not automatically support keyboard operation or communicate checked state. This made an important call-quality workflow mouse-dependent.

Benefit to users

The entire checklist can now be completed and reset with standard keyboard controls.

9. Accessible Notes Log dialog

What changed

The Notes Log chooser now:

- Identifies itself as a modal dialog.
- Has an accessible title and description.
- Places initial focus on a useful action.
- Keeps Tab navigation inside the dialog.
- Supports Escape and backdrop cancellation.

- Returns focus to the button that opened it.

Why this was needed

The former prompt looked like a modal but did not behave like one for keyboard or screen-reader users. Focus could move behind it or become lost after closing.

Benefit to users

Opening and closing the Notes Log chooser is predictable, including for people who do not use a mouse.

10. Better behavior in narrow windows

What changed

- At narrow desktop widths, the QA checklist becomes a slide-out overlay.
- A backdrop separates the checklist from the form.
- Forms switch to one column before controls become cramped.
- Text used for labels, checklist instructions, errors, reminders, and statuses was made more readable.
- Visible focus outlines were added to interactive controls.
- Reduced-motion preferences are respected.

Why this was needed

The fixed-width checklist could leave too little room for the main form. Some supporting text was also unusually small, and animated transitions did not account for users who prefer reduced motion.

Benefit to users

The application remains usable at smaller window sizes, browser zoom levels, and a wider range of accessibility preferences.

Technical improvements

11. Explicit application state

What changed

Hidden custom properties such as `_plain`, `_rows`, and `_userEdited` were removed from DOM elements. Summary text, preview text, and user-edited tracking now live in explicit JavaScript state.

Why this was needed

Attaching private data directly to page elements creates invisible dependencies. Re-rendering an element can silently discard that state, and future maintainers must know which nonstandard properties happen to exist.

Technical approach

- Summary rows and plain text use dedicated state variables.
- Note preview text uses a dedicated state variable.
- User-edited controls are tracked with a `WeakSet`.
- Shared field propagation checks this explicit dirty state before replacing a value.

12. Less duplicated configuration

What changed

Both note and escalation forms now use one EMS option list.

Why this was needed

Two identical lists could drift apart when a system was added, removed, or renamed.

Benefit

One update now changes both workflows consistently.

13. More predictable initialization

What changed

The initial note form is rendered once, after all template data and functions have been defined.

Why this was needed

Event-based initialization could run at different times in a direct file, embedded environment, or automated test. Duplicate rendering could replace a control while it had focus.

Benefit

Startup is deterministic and easier to test.

Testing and verification

Automated regression coverage

A focused Node.js and jsdom suite was added at:

`development/programs/tests/all-in-one-v7.test.js`

Run it from the repository root with:

```
npm run test:all-in-one
```

The suite currently contains **9 tests** covering:

1. Safe rendering of HTML-like input and complete escalation Undo.
2. Point Change conditional New Board validation.
3. Isolation of Notes Clear/Undo and DAE routing reset behavior.
4. Clipboard rejection and prevention of incorrect Notes Log writes.
5. Rendering and validation of every escalation type.
6. Generated labels, errors, ARIA state, and focus recovery.
7. QA checklist state, responsive overlay behavior, and Escape handling.
8. Notes Log dialog behavior and a mocked file write.
9. Every notes template, including labels, generated previews, and unresolved placeholder checks.

Final verification results

Check	Result
Automated tests	9 passed, 0 failed
Editor diagnostics	No errors in the app, tests, package file, or documentation
Whitespace validation	`git diff --check` passed
Supported packaging	Single directly openable HTML file retained

Supported environment and limitations

Supported browser

The general notes and escalation workflows are intended for current desktop browsers. The **Notes Log** file picker specifically requires a current version of **Google Chrome or Microsoft Edge** because it uses the File System Access API.

Data handling

Technician, store, work-order, and note values remain in the open page only. This tool does not persist those values to `localStorage`. A Notes Log is written only after the user explicitly selects a file and grants browser permission.

Retained design decisions

- The application remains one directly openable HTML file.
- Existing business templates and generated wording were preserved.
- External Google Fonts were retained with browser fallback fonts.

- The current dark, compact visual identity was preserved.
- The Notes Log continues to rewrite the selected text file when appending; a warning appears for files larger than 5 MB.

Files changed

File	Purpose
`development/programs/All_In_1_v7.html`	Production application behavior, accessibility, layout, and browser integration
`development/programs/tests/all-in-one-v7.test.js`	Automated regression coverage
`package.json`	Adds the `test:all-in-one` command
`README.md`	Documents the standalone tool, browser requirement, privacy behavior, and test command

Summary

The improvement work addressed immediate security and data-loss risks first, then strengthened validation, browser operations, accessibility, responsive behavior, internal state management, and automated testing.

For everyday users, the result is a tool that gives clearer feedback, protects work more reliably, and supports more ways of interacting with the page. For maintainers, the result is safer rendering, more explicit state, less duplicated configuration, and a regression suite that exercises every notes template and escalation type.

Document status: Final
Application reviewed: QA Notes & Escalation Tool, `All_In_1_v7.html`
Verification baseline: 9 automated tests passing on September 12, 2026